ISIT312/ISIT912 Big Data Management

Spring 2025

Hadoop and HDFS Practice

Objective: After this practice, you will get familiar with using Linux shell to interact with Hadoop.

Warning: To copy the fragments of code from this document to your working `Terminal` use `evince` pdf viewer.

Software Installation and Setup.

If you use a system installed in 3.230 i.e. a laboratory room for ISIT312/912, then you do not need to install VirtualBox.

If you use your system (laptop) then start from installation of the latest version of VirtualBox and the extension pack (optional). The distribution packages are available at https://www.virtualbox.org/wiki/Downloads for Windows, MacOS (Intel processor only) and Linux.

If you use your system (laptop) then you must copy a file `BigData-Ubuntu22.04.4-Hadoop3.3.6-P3-Spark3.5.1.ova` file located at `C:\Program Files (x86)\ISIT312 and ISIT912` (if you cannot find the file in this location please ask a tutor about correct location) folder to your system. You can also download the file from Moodle.

Next, start VirtualBox and import the file. An instruction is in

https://docs.oracle.com/cd/E26217_01/E26796/html/qs-import-vm.html

or you can just double-click at the `ova` file.

If you use a system installed in 3.230 then start VirtualBox and import a file `BigData-Ubuntu22.04.4-Hadoop3.3.6-P3-Spark3.5.1.ova` file located at `C:\VM\ISIT312` folder into VirtualBox. An instruction is in

https://docs.oracle.com/cd/E26217_01/E26796/html/qs-import-vm.html

or you can just double-click at the `ova` file.

After the import is completed, a virtual machine named `BigData-Ubuntu22.04.4-Hadoop3.3.6-P3-Spark3.5.1` appears in the VirtualBox.

Laboratory Instructions

Start a virtual machine `BigData-Ubuntu22.04.4-Hadoop3.3.6-P3-Spark3.5.1`

Note, that both the account name and the password are: `bigdata`

(0) Start Shell

After you log on the Ubuntu system with a user name `bigdata` and password `bigdata`, start a `Terminal` window with `Ctrl + Alt + T` or use the fifth from top icon located at a vertical stripe (sidebar) on the left-hand side of the screen.

The documents LinuxCommandLineCheatSheet.pdf and Efficient-Linux-at-the-Command-Line-ch4.pdf available through Readings link in Resources section in linux folder on Moodle contain more information on how to use Linux Shell in Terminal window.

You can use the Terminal window to interact with Hadoop. A simple hint that may make your life much easier is to use "Up" and "Down" keyboard keys to navigate through the commands already processed in Terminal window.

Now you can interact with Hadoop.

(1) Hadoop files and scripts

Process the following command to have a look at what is contained in the $HADOOP_HOME:

```
ls $HADOOP_HOME       # view Hadoop homefolder
ls $HADOOP_HOME/bin   # view the "bin" folder
ls $HADOOP_HOME/sbin  # view the "sbin" folder
```

The bin and sbin folders contain the programs and shell scripts for initialization and management of Hadoop.

(2) Hadoop initialisation

Now you can start Hadoop. Process a shell script start-all.sh in Terminal window in the following way:

```
$HADOOP_HOME/sbin/start-all.sh
```

The system will display few warnings and the following messages:

```
Starting namenodes on [localhost]
Starting datanodes
Starting secondary namenodes [bigdata-VirtualBox]
Starting resourcemanager
Starting nodemanagers
```

After you ignore the warning messages you can view the running processes with a command:

```
jps
```

The following results should be returned (note that the process numbers may be different):

```
819 NodeManager
2501 SecondaryNameNode
2310 DataNode
4391 Jps
2187 NameNode
2700 ResourceManager
```

(3) HDFS Shell commands

To create a folder myfolder in a home folder of bigdata user in HDFS use hdfs program.

```
$HADOOP_HOME/bin/hdfs dfs -mkdir myfolder
```

A command option -mkdir creates a new fiolder myfolder in a home folder (/user/bigdata) of bigdata user. To verify if the results use hdfs program.

```
$HADOOP_HOME/bin/hdfs dfs -ls
```

You can also use absolute path from a root folder of HDFS file system.

```
$HADOOP_HOME/bin/hdfs dfs -ls /user/bigdata
```

A program `hdfs` can be used as a shell command with the following parameters.

```
hdfs {OPTIONS} SUBCOMMAND [SUBCOMMAND OPTION]
```

It means that there is no need to prefix `hdfs` command with `$HADOOP_HOME/bin`. You can get information about `[OPTIONS]` and `SUBCOMMAND` from the following processing of `hdfs`.

```
hdfs help
```

In the future we shall use only one subcommand `dfs`. You can get more information about `[SUBCOMMAND OPTION]`s of subcommand `dfs` in the following way.

```
hdfs dfs
```

Now, we try a number of typical options of a subcommand `dfs`.

First, we copy a file from the local filesystem to HDFS. The following command copies all files with an extension `.txt` located in `$HADOOP_HOME` to the earlier created folder `myfolder` located in HDFS:

```
hdfs dfs -put $HADOOP_HOME/*.txt myfolder
```

Next, we list files in the root folder in HDFS and `myfolder` folder in HDFS.

```
hdfs dfs -ls /
hdfs dfs -ls myfolder
```

Next, we list a file located in `myfolder` in HDFS.

```
hdfs dfs -cat myfolder/README.txt
```

Next, we copy a file from HDFS to the local filesystem:

```
hdfs dfs -copyToLocal myfolder/README.txt /home/bigdata/Desktop
```

We can verify the results with

```
ls /home/bigdata/Desktop
```

Next, we remove a file located in a folder `myfolder` in HDFS.

```
hdfs dfs -rm myfolder/README.txt
```

The majority of subcommand options of subcommand `dfs` are identical to the shell commands of Unix/Linux operating systems. You can find more information about Linux shell command either in `Efficient-Linux-at-the-Command-Line-ch4.pdf` or in `LinuxCommandLineCheatSheet.pdf` section `5 File and Directory Commands` available on Moodle through the `Readings` link included in `Resources` section. To download the files navigate to `linux` folder.

(4) HDFS Graphical User Interface

Open the Firefox Web Browser, go to `localhost:9870`. You will see Overview of `localhost:9000`. This is the location of the HDFS. It is specified in a configuration file named `core-site.xml`.

Check this file in `$HADOOP_HOME/etc/hadoop`, which contains Hadoop's configuration files.

You can view a file `core-site.xml` in Terminal:

```
cat $HADOOP_HOME/etc/hadoop/core-site.xml
```

Browse the web UI (e.g., you can see the location of the Datanode). Go to "Utilities" and then to "Browser the file system". Check the `.txt` files uploaded to HDFS previously. Note that the root folder of `bigdata` is in the `user` folder.

To view all root folders in HDFS in Terminal, you can also enter the following command in Terminal:

```
hdfs dfs -ls /
```

(5) HDFS Python Interface

A Python package `Snakebite` created by Spotify can be used to access HDFS programmatically from Python applications. Protocol buffer messages are used to communicate directly with the `NameNode`. `Snakebite` package also includes its own command-line interface to HDFS.

In this step copy each Python application listed below into a clipboard, start `gedit` text editor, paste it into a blank text editor window and save it in a file with an extension `py`. We start from a sample application of `Snakebite` that lists the contents of `/user` folder in HDFS.

```
#!/usr/bin/env python3

from snakebite.client import Client

client = Client('localhost', 9000)
for x in client.ls(['/user']):
      print(x)
```

Copy an application into a clipboard, paste it into `gedit` window and save it in a file `ls.py`.

Then change user's access permission to a file `ls.py` with

```
chmod u+x ls.py
```

To process the application use

```
./ls.py
```

in the Terminal window.

The results are listed below.

```
{'file_type': 'd', 'permission': 493, 'path': '/user/bigdata',
'length': 0, 'owner': 'bigdata', 'group': 'supergroup',
'block_replication': 0, 'modification_time': 1714891461351,
'access_time': 0, 'blocksize': 0}
```

The example above uses `ls` method that returns a generator that need to be consumed to list the names of files and folders included in a folder `/user`.

Compare the results listed above with

```
hdfs dfs -ls /user
```

A sample application of `Snakebite` that lists the names of files in `/user/bigdata/myfolder` folder in HDFS is given below.

```
#!/usr/bin/env python3

from snakebite.client import Client

client = Client('localhost', 9000)
for x in client.ls(['/user/bigdata/myfolder']):
     print(x['path'])
```

Implement the application given above and test it with few paths different from `/user/bigdata/myfolder`.

A sample application of `Snakebite` that that copies a file `NOTICE.txt` from HDFS to a local file system is given below.

```
#!/usr/bin/env python3

from snakebite.client import Client

client = Client('localhost', 9000)
for x in \
client.copyToLocal(['/user/bigdata/myfolder/NOTICE.txt'],'.'):
     print(x)
```

Implement the application given above and test it with few files located in HDFS.

A sample application of `Snakebite` that that that lists a file located in HDFS is given below.

```
#!/usr/bin/env python3

from snakebite.client import Client

client = Client('localhost', 9000)
for file in client.cat(['/user/bigdata/myfolder/NOTICE.txt']):
     for y in file:
            print(y)
```

Implement the application given above and test it with the few files located in HDFS.

A sample application of `Snakebite` that that that lists several files located in HDFS is given below.

```
#!/usr/bin/env python3

from snakebite.client import Client

client = Client('localhost', 9000)

files = []
for x in client.ls(['/user/bigdata/myfolder/*txt']):
        if x['file_type'] == 'f':
                files.append(x['path'])

for cat in client.text(files):
      print(cat)
```

Implement the application given above and test it with the few files located in HDFS.

Use a text editor (either `gedit` or `nano`) to create a file `numbers.txt` contains a sequence of numbers. You can create any sequence of numbers separated and followed with blank lines.

```
23
56

78
23
12
```

Next, upload the file to HDFS

```
hdfs dfs -put numbers.txt /user/bigdata
```

Use a text editor to create a file `filter.py` with the following Python program.

```
#!/usr/bin/env python3

import sys

for number in sys.stdin:
     if (number != "\n") and int(number) > 50:
                print(int(number))
```

Then change user's access permission to a file `filter.py` with

```
chmod u+x filter.py
```

The program uses Hadoop streaming feature (well explained in the next lab) to read a file `numbers.txt` from HDFS, to filter out the numbers less or equal to `50` and to save the results in HDFS.

Make sure that your file `filter.py` is located in `/home/bigdata` folder. Then you can process the program with Hadoop streaming feature in the following way.

```
$HADOOP_HOME/bin/hadoop jar \
  $HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-3.3.6.jar \
```

```
    -D mapred.reduce.tasks=0 \
    -input /user/bigdata/numbers.txt \
    -output /user/bigdata/output \
    -mapper /home/bigdata/filter.py
```

The results can be found in HDFS with

```
hdfs dfs -ls /user/bigdata/output
```

and later on, listed with

```
hdfs dfs -cat /user/bigdata/output/part*
```

(6) Shut down Hadoop

When finishing your practice with Hadoop, it is good practice to terminate the Hadoop daemons before turning off the virtual machine.

You can stop all Hadoop processes with a shell script `stop-all.sh`.

```
$HADOOP_HOME/sbin/stop-all.sh
```

(7) Make a typescript of information in `Terminal` (optional)

The `script` command to record everything printed in your `Terminal`.

```
script a-file-name-you-want-to-save-the-typescript-to.txt
<work with your Terminal..>
exit
```

To see the result check the contents of `a-file-name-you-want-to-save-the-typescript-to.txt`. It is a text file and you can open it with either `gedit` or `nano` text editors.

Appendix

```
Usage: hdfs dfs [generic options]
        [-appendToFile [-n] <localsrc> ... <dst>]
        [-cat [-ignoreCrc] <src> ...]
        [-checksum [-v] <src> ...]
        [-chgrp [-R] GROUP PATH...]
        [-chmod [-R] <MODE[,MODE]... | OCTALMODE> PATH...]
        [-chown [-R] [OWNER][:[GROUP]] PATH...]
        [-concat <target path> <src path> <src path> ...]
        [-copyFromLocal [-f] [-p] [-l] [-d] [-t <thread count>] [-q <thread pool queue size>]
<localsrc> ... <dst>]
        [-copyToLocal [-f] [-p] [-crc] [-ignoreCrc] [-t <thread count>] [-q <thread pool queue
size>] <src> ... <localdst>]
```

```
        [-count [-q] [-h] [-v] [-t [<storage type>]] [-u] [-x] [-e] [-s] <path> ...]

        [-cp [-f] [-p | -p[topax]] [-d] [-t <thread count>] [-q <thread pool queue size>] <src> ...
<dst>]

        [-createSnapshot <snapshotDir> [<snapshotName>]]

        [-deleteSnapshot <snapshotDir> <snapshotName>]

        [-df [-h] [<path> ...]]

        [-du [-s] [-h] [-v] [-x] <path> ...]

        [-expunge [-immediate] [-fs <path>]]

        [-find <path> ... <expression> ...]

        [-get [-f] [-p] [-crc] [-ignoreCrc] [-t <thread count>] [-q <thread pool queue size>]
<src> ... <localdst>]

        [-getfacl [-R] <path>]

        [-getfattr [-R] {-n name | -d} [-e en] <path>]

        [-getmerge [-nl] [-skip-empty-file] <src> <localdst>]

        [-head <file>]

        [-help [cmd ...]]

        [-ls [-C] [-d] [-h] [-q] [-R] [-t] [-S] [-r] [-u] [-e] [<path> ...]]

        [-mkdir [-p] <path> ...]

        [-moveFromLocal [-f] [-p] [-l] [-d] <localsrc> ... <dst>]

        [-moveToLocal <src> <localdst>]

        [-mv <src> ... <dst>]

        [-put [-f] [-p] [-l] [-d] [-t <thread count>] [-q <thread pool queue size>] <localsrc> ...
<dst>]

        [-renameSnapshot <snapshotDir> <oldName> <newName>]

        [-rm [-f] [-r|-R] [-skipTrash] [-safely] <src> ...]

        [-rmdir [--ignore-fail-on-non-empty] <dir> ...]

        [-setfacl [-R] [{-b|-k} {-m|-x <acl_spec>} <path>]|[--set <acl_spec> <path>]]

        [-setfattr {-n name [-v value] | -x name} <path>]

        [-setrep [-R] [-w] <rep> <path> ...]

        [-stat [format] <path> ...]

        [-tail [-f] [-s <sleep interval>] <file>]

        [-test -[defswrz] <path>]

        [-text [-ignoreCrc] <src> ...]

        [-touch [-a] [-m] [-t TIMESTAMP (yyyyMMdd:HHmmss) ] [-c] <path> ...]

        [-touchz <path> ...]

        [-truncate [-w] <length> <path> ...]

        [-usage [cmd ...]]


-appendToFile [-n] <localsrc> ... <dst> :

  Appends the contents of all the given local files to the given dst file. The dst

  file will be created if it does not exist. If <localSrc> is -, then the input is

  read from stdin. Option -n represents that use NEW_BLOCK create flag to append
```

file.

-cat [-ignoreCrc] <src> ... :

Fetch all files that match the file pattern <src> and display their content on stdout.

-checksum [-v] <src> ... :

Dump checksum information for files that match the file pattern <src> to stdout. Note that this requires a round-trip to a datanode storing each block of the file, and thus is not efficient to run on a large number of files. The checksum of a file depends on its content, block size and the checksum algorithm and parameters used for creating the file.

-chgrp [-R] GROUP PATH... :

This is equivalent to -chown ... :GROUP ...

-chmod [-R] <MODE[,MODE]... | OCTALMODE> PATH... :

Changes permissions of a file. This works similar to the shell's chmod command with a few exceptions.

-R	modifies the files recursively. This is the only option currently supported.
<MODE>	Mode is the same as mode used for the shell's command. The only letters recognized are 'rwxXt', e.g. +t,a+r,g-w,+rwx,o=r.
<OCTALMODE>	Mode specifed in 3 or 4 digits. If 4 digits, the first may be 1 or 0 to turn the sticky bit on or off, respectively. Unlike the shell command, it is not possible to specify only part of the mode, e.g. 754 is same as u=rwx,g=rx,o=r.

If none of 'augo' is specified, 'a' is assumed and unlike the shell command, no umask is applied.

-chown [-R] [OWNER][:[GROUP]] PATH... :

Changes owner and group of a file. This is similar to the shell's chown command with a few exceptions.

-R modifies the files recursively. This is the only option currently supported.

If only the owner or group is specified, then only the owner or group is modified. The owner and group names may only consist of digits, alphabet, and

```
  any of [-_./@a-zA-Z0-9]. The names are case sensitive.

  WARNING: Avoid using '.' to separate user name and group though Linux allows it.
  If user names have dots in them and you are using local file system, you might
  see surprising results since the shell command 'chown' is used for local files.

-concat <target path> <src path> <src path> ... :
  Concatenate existing source files into the target file. Target file and source
  files should be in the same directory.

-copyFromLocal [-f] [-p] [-l] [-d] [-t <thread count>] [-q <thread pool queue size>] <localsrc> ...
<dst> :
  Identical to the -put command.

-copyToLocal [-f] [-p] [-crc] [-ignoreCrc] [-t <thread count>] [-q <thread pool queue size>]
<src> ... <localdst> :
  Identical to the -get command.

-count [-q] [-h] [-v] [-t [<storage type>]] [-u] [-x] [-e] [-s] <path> ... :
  Count the number of directories, files and bytes under the paths
  that match the specified file pattern.  The output columns are:
  DIR_COUNT FILE_COUNT CONTENT_SIZE PATHNAME
  or, with the -q option:
  QUOTA REM_QUOTA SPACE_QUOTA REM_SPACE_QUOTA
        DIR_COUNT FILE_COUNT CONTENT_SIZE PATHNAME
  The -h option shows file sizes in human readable format.
  The -v option displays a header line.
  The -x option excludes snapshots from being calculated.
  The -t option displays quota by storage types.
  It should be used with -q or -u option, otherwise it will be ignored.
  If a comma-separated list of storage types is given after the -t option,
  it displays the quota and usage for the specified types.
  Otherwise, it displays the quota and usage for all the storage
  types that support quota. The list of possible storage types(case insensitive):
  ram_disk, ssd, disk and archive.
  It can also pass the value '', 'all' or 'ALL' to specify all the storage types.
  The -u option shows the quota and
  the usage against the quota without the detailed content summary.The -e option
  shows the erasure coding policy.The -s option shows snapshot counts.

-cp [-f] [-p | -p[topax]] [-d] [-t <thread count>] [-q <thread pool queue size>] <src> ... <dst> :
  Copy files that match the file pattern <src> to a destination. When copying
```

multiple files, the destination must be a directory.

Flags :

-p[topax]	Preserve file attributes [topx] (timestamps, ownership, permission, ACL, XAttr). If -p is specified with no arg, then preserves timestamps, ownership, permission. If -pa is specified, then preserves permission also because ACL is a super-set of permission. Determination of whether raw namespace extended attributes are preserved is independent of the -p flag.
-f	Overwrite the destination if it already exists.
-d	Skip creation of temporary file(<dst>._COPYING_).
-t <thread count>	Number of threads to be used, default is 1.
-q <thread pool queue size>	Thread pool queue size to be used, default is 1024.

-createSnapshot <snapshotDir> [<snapshotName>] :

Create a snapshot on a directory

-deleteSnapshot <snapshotDir> <snapshotName> :

Delete a snapshot from a directory

-df [-h] [<path> ...] :

Shows the capacity, free and used space of the filesystem. If the filesystem has multiple partitions, and no path to a particular partition is specified, then the status of the root partitions will be shown.

-h Formats the sizes of files in a human-readable fashion rather than a number of bytes.

-du [-s] [-h] [-v] [-x] <path> ... :

Show the amount of space, in bytes, used by the files that match the specified file pattern. The following flags are optional:

-s Rather than showing the size of each individual file that matches the pattern, shows the total (summary) size.

-h Formats the sizes of files in a human-readable fashion rather than a number of bytes.

-v option displays a header line.

-x Excludes snapshots from being counted.

```
  Note that, even without the -s option, this only shows size summaries one level
  deep into a directory.

  The output is in the form
        size    disk space consumed     name(full path)

-expunge [-immediate] [-fs <path>] :
  Delete files from the trash that are older than the retention threshold

-find <path> ... <expression> ... :
  Finds all files that match the specified expression and
  applies selected actions to them. If no <path> is specified
  then defaults to the current working directory. If no
  expression is specified then defaults to -print.

  The following primary expressions are recognised:
    -name pattern
    -iname pattern
      Evaluates as true if the basename of the file matches the
      pattern using standard file system globbing.
      If -iname is used then the match is case insensitive.

    -print
    -print0
      Always evaluates to true. Causes the current pathname to be
      written to standard output followed by a newline. If the -print0
      expression is used then an ASCII NULL character is appended rather
      than a newline.

  The following operators are recognised:
    expression -a expression
    expression -and expression
    expression expression
      Logical AND operator for joining two expressions. Returns
      true if both child expressions return true. Implied by the
      juxtaposition of two expressions and so does not need to be
      explicitly specified. The second expression will not be
      applied if the first fails.
```

```
-get [-f] [-p] [-crc] [-ignoreCrc] [-t <thread count>] [-q <thread pool queue size>] <src> ...
<localdst> :

  Copy files that match the file pattern <src> to the local name. <src> is kept.

  When copying multiple files, the destination must be a directory.

  Flags:

  -p                            Preserves timestamps, ownership and the mode.
  -f                            Overwrites the destination if it already exists.
  -crc                          write CRC checksums for the files downloaded.
  -ignoreCrc                    Skip CRC checks on the file(s) downloaded.
  -t <thread count>             Number of threads to be used, default is 1.
  -q <thread pool queue size>   Thread pool queue size to be used, default is
                                1024.

-getfacl [-R] <path> :

  Displays the Access Control Lists (ACLs) of files and directories. If a
  directory has a default ACL, then getfacl also displays the default ACL.

  -R      List the ACLs of all files and directories recursively.
  <path>  File or directory to list.

-getfattr [-R] {-n name | -d} [-e en] <path> :

  Displays the extended attribute names and values (if any) for a file or
  directory.

  -R              Recursively list the attributes for all files and directories.
  -n name         Dump the named extended attribute value.
  -d              Dump all extended attribute values associated with pathname.
  -e <encoding>   Encode values after retrieving them.Valid encodings are "text",
                  "hex", and "base64". Values encoded as text strings are enclosed
                  in double quotes ("), and values encoded as hexadecimal and
                  base64 are prefixed with 0x and 0s, respectively.
  <path>          The file or directory.

-getmerge [-nl] [-skip-empty-file] <src> <localdst> :

  Get all the files in the directories that match the source file pattern and
  merge and sort them to only one file on local fs. <src> is kept.

  -nl               Add a newline character at the end of each file.
  -skip-empty-file  Do not add new line character for empty file.
```

-head <file> :

  Show the first 1KB of the file.

-help [cmd ...] :

  Displays help for given command or all commands if none is specified.

-ls [-C] [-d] [-h] [-q] [-R] [-t] [-S] [-r] [-u] [-e] [<path> ...] :

  List the contents that match the specified file pattern. If path is not
  specified, the contents of /user/<currentUser> will be listed. For a directory a
  list of its direct children is returned (unless -d option is specified).

  Directory entries are of the form:
        permissions - userId groupId sizeOfDirectory(in bytes)
  modificationDate(yyyy-MM-dd HH:mm) directoryName

  and file entries are of the form:
        permissions numberOfReplicas userId groupId sizeOfFile(in bytes)
  modificationDate(yyyy-MM-dd HH:mm) fileName

    -C Display the paths of files and directories only.
    -d Directories are listed as plain files.
    -h Formats the sizes of files in a human-readable fashion
        rather than a number of bytes.
    -q Print ? instead of non-printable characters.
    -R Recursively list the contents of directories.
    -t Sort files by modification time (most recent first).
    -S Sort files by size.
    -r Reverse the order of the sort.
    -u Use time of last access instead of modification for
        display and sorting.
    -e Display the erasure coding policy of files and directories.

-mkdir [-p] <path> ... :

  Create a directory in specified location.

  -p Do not fail if the directory already exists

-moveFromLocal [-f] [-p] [-l] [-d] <localsrc> ... <dst> :

  Same as -put, except that the source is deleted after it's copied
  and -t option has not yet implemented.

```
-moveToLocal <src> <localdst> :
  Not implemented yet

-mv <src> ... <dst> :
  Move files that match the specified file pattern <src> to a destination <dst>.
  When moving multiple files, the destination must be a directory.

-put [-f] [-p] [-l] [-d] [-t <thread count>] [-q <thread pool queue size>] <localsrc> ... <dst> :
  Copy files from the local file system into fs. Copying fails if the file already
  exists, unless the -f flag is given.
  Flags:

  -p                            Preserves timestamps, ownership and the mode.
  -f                            Overwrites the destination if it already exists.
  -t <thread count>             Number of threads to be used, default is 1.
  -q <thread pool queue size>   Thread pool queue size to be used, default is
                                1024.
  -l                            Allow DataNode to lazily persist the file to disk.
                                Forces replication factor of 1. This flag will
                                result in reduced durability. Use with care.
  -d                            Skip creation of temporary file(<dst>._COPYING_).

-renameSnapshot <snapshotDir> <oldName> <newName> :
  Rename a snapshot from oldName to newName

-rm [-f] [-r|-R] [-skipTrash] [-safely] <src> ... :
  Delete all files that match the specified file pattern. Equivalent to the Unix
  command "rm <src>"

  -f          If the file does not exist, do not display a diagnostic message or
              modify the exit status to reflect an error.
  -[rR]       Recursively deletes directories.
  -skipTrash  option bypasses trash, if enabled, and immediately deletes <src>.
  -safely     option requires safety confirmation, if enabled, requires
              confirmation before deleting large directory with more than
              <hadoop.shell.delete.limit.num.files> files. Delay is expected when
              walking over large directory recursively to count the number of
              files to be deleted before the confirmation.

-rmdir [--ignore-fail-on-non-empty] <dir> ... :
  Removes the directory entry specified by each directory argument, provided it is
```

```
  empty.

-setfacl [-R] [{-b|-k} {-m|-x <acl_spec>} <path>]|[--set <acl_spec> <path>] :
  Sets Access Control Lists (ACLs) of files and directories.
  Options:

  -b          Remove all but the base ACL entries. The entries for user, group
              and others are retained for compatibility with permission bits.
  -k          Remove the default ACL.
  -R          Apply operations to all files and directories recursively.
  -m          Modify ACL. New entries are added to the ACL, and existing entries
              are retained.
  -x          Remove specified ACL entries. Other ACL entries are retained.
  --set       Fully replace the ACL, discarding all existing entries. The
              <acl_spec> must include entries for user, group, and others for
              compatibility with permission bits. If the ACL spec contains only
              access entries, then the existing default entries are retained. If
              the ACL spec contains only default entries, then the existing
              access entries are retained. If the ACL spec contains both access
              and default entries, then both are replaced.
  <acl_spec>  Comma separated list of ACL entries.
  <path>      File or directory to modify.

-setfattr {-n name [-v value] | -x name} <path> :
  Sets an extended attribute name and value for a file or directory.

  -n name   The extended attribute name.
  -v value  The extended attribute value. There are three different encoding
            methods for the value. If the argument is enclosed in double quotes,
            then the value is the string inside the quotes. If the argument is
            prefixed with 0x or 0X, then it is taken as a hexadecimal number. If
            the argument begins with 0s or 0S, then it is taken as a base64
            encoding.
  -x name   Remove the extended attribute.
  <path>    The file or directory.

-setrep [-R] [-w] <rep> <path> ... :
  Set the replication level of a file. If <path> is a directory then the command
  recursively changes the replication factor of all files under the directory tree
  rooted at <path>. The EC files will be ignored here.
```

```
  -w  It requests that the command waits for the replication to complete. This
      can potentially take a very long time.
  -R  It is accepted for backwards compatibility. It has no effect.

-stat [format] <path> ... :
  Print statistics about the file/directory at <path>
  in the specified format. Format accepts permissions in
  octal (%a) and symbolic (%A), filesize in
  bytes (%b), type (%F), group name of owner (%g),
  name (%n), block size (%o), replication (%r), user name
  of owner (%u), access date (%x, %X).
  modification date (%y, %Y).
  %x and %y show UTC date as "yyyy-MM-dd HH:mm:ss" and
  %X and %Y show milliseconds since January 1, 1970 UTC.
  If the format is not specified, %y is used by default.

-tail [-f] [-s <sleep interval>] <file> :
  Show the last 1KB of the file.

  -f  Shows appended data as the file grows.
  -s  With -f , defines the sleep interval between iterations in milliseconds.

-test -[defswrz] <path> :
  Answer various questions about <path>, with result via exit status.
    -d  return 0 if <path> is a directory.
    -e  return 0 if <path> exists.
    -f  return 0 if <path> is a file.
    -s  return 0 if file <path> is greater         than zero bytes in size.
    -w  return 0 if file <path> exists         and write permission is granted.
    -r  return 0 if file <path> exists         and read permission is granted.
    -z  return 0 if file <path> is         zero bytes in size, else return 1.

-text [-ignoreCrc] <src> ... :
  Takes a source file and outputs the file in text format.
  The allowed formats are zip and TextRecordInputStream and Avro.

-touch [-a] [-m] [-t TIMESTAMP (yyyyMMdd:HHmmss) ] [-c] <path> ... :
  Updates the access and modification times of the file specified by the <path> to
  the current time. If the file does not exist, then a zero length file is created
  at <path> with current time as the timestamp of that <path>.
  -a Change only the access time
```

```
  -m Change only the modification time

  -t TIMESTAMP Use specified timestamp instead of current time

   TIMESTAMP format yyyyMMdd:HHmmss

  -c Do not create any files


-touchz <path> ... :

  Creates a file of zero length at <path> with current time as the timestamp of

  that <path>. An error is returned if the file exists with non-zero length


-truncate [-w] <length> <path> ... :

  Truncate all files that match the specified file pattern to the specified

  length.


  -w  Requests that the command wait for block recovery to complete, if

      necessary.


-usage [cmd ...] :

  Displays the usage for given command or all commands if none is specified.


Generic options supported are:

-conf <configuration file>        specify an application configuration file

-D <property=value>               define a value for a given property

-fs <file:///|hdfs://namenode:port> specify default filesystem URL to use, overrides 'fs.defaultFS'
property from configurations.

-jt <local|resourcemanager:port>  specify a ResourceManager

-files <file1,...>                specify a comma-separated list of files to be copied to the map
reduce cluster

-libjars <jar1,...>               specify a comma-separated list of jar files to be included in the
classpath

-archives <archive1,...>          specify a comma-separated list of archives to be unarchived on the
compute machines


The general command line syntax is:

command [genericOptions] [commandOptions]
```